

```
-- Joins:-
-- a join is used to combine rows from two or more tables based on a related
column.

-- students, courses, enrollments(Junction table).

create database if not exists joinsDB;
use joinsDB;

create table students(s_id int primary key auto_increment, s_name varchar(225),
s_age int) auto_increment = 101;

create table courses(c_id int primary key auto_increment, c_name varchar(225))
auto_increment = 1001;

create table enrollments (s_id int, c_id int, marks decimal(5,2),
constraint fk_student_enrollments foreign key(s_id) references students(s_id)
on delete cascade on update cascade,
constraint fk_courses_enrollments foreign key(c_id) references courses(c_id)
on delete cascade on update cascade);
```

```
INSERT INTO students (s_name, s_age) VALUES
('Aman Sharma', 20),
('Priya Verma', 21),
('Rahul Singh', 22),
('Sneha Kapoor', 19),
('Arjun Mehta', 23),
('Kavya Nair', 20),
('Rohit Gupta', 24),
('Ananya Das', 21),
('Vikas Yadav', 22),
('Meera Joshi', 20);
```

```
INSERT INTO courses (c_name) VALUES
('Java Programming'),
('Database Management'),
('Web Development'),
('Data Structures'),
('Operating Systems');
```

```
INSERT INTO enrollments (s_id, c_id, marks) VALUES
(101, 1001, 85.50),
(101, 1002, 78.00),

(102, 1001, 88.00),
```

```
(102, 1003, 91.50),  
  
(103, 1002, 72.00),  
(103, 1004, 80.00),  
  
(104, 1003, 89.00),  
(104, 1005, 76.50),  
  
(105, 1001, 90.00),  
(105, 1004, 84.00),  
  
(106, 1002, 67.50),  
(106, 1003, 73.00),  
  
(107, 1004, 88.50),  
(107, 1005, 92.00),  
  
(108, 1001, 79.00),  
(108, 1002, 81.00),  
  
(109, 1003, 86.00),  
(109, 1005, 74.00),  
  
(110, 1002, 69.00),  
(110, 1004, 95.00);
```

```
-- syntax for join  
select column1, column2  
from table1  
join table2  
on table1.column1 = table2.column2;
```

```
-- Types of joins:-
```

```
-- 1) inner join(simple join)  
-- 2) Left join(left outer join)  
-- 3) Right join(Right outer join)  
-- 4) Cross join  
-- 5) Self join(conceptual, not a seprate join)  
-- 6) Full outer join(Workaround required, not supported by MySQL)  
-- 7) left null join  
-- 8) right null join
```

```
-- 1) Inner join(80%):-
```

```
-- returns only matching rows from both tables.  
-- If no match --> Row is excluded
```

```
-- with table name
select *
from students
inner join enrollments
on students.s_id = enrollments.s_id;
-- using aliases for table name
select *
from students as s
inner join enrollments as e
on s.s_id = e.s_id;
```

```
select e.s_id, s.s_name, e.c_id, e.marks, c.c_name
from students s
inner join enrollments e
on s.s_id = e.s_id
join courses c
on c.c_id = e.c_id;
```

```
-- 2) Left join(left outer join)(15%):-
-- left join returns all the rows from the left table
-- and matching rows from the right table.
-- if no match then populate the right table with null.
```

```
select * from students;
select * from courses;
select * from enrollments;
```

```
-- display all the students enrollments regardless of their
-- enrollment into any course.
```

```
select *
from students s
left join enrollments e
on s.s_id = e.s_id
left join courses c
on c.c_id = e.c_id;
```

```
-- display all the courses regardless if they
-- have an enrollment or not.
```

```
select *
from courses c
left join enrollments e
on c.c_id = e.c_id
left join students s
on s.s_id = e.s_id;
```

```
-- 3) Left null join:-
```

```

-- left null join is used to find the unique records
-- to the left of the table and the one that are
-- not overlapping withg the right table
select *
from students s
left join enrollments e
on s.s_id = e.s_id
left join courses c
on c.c_id = e.c_id
where e.s_id is null;

```

```

-- 4) Right join(Right outer join)(rarely used):-
-- all rows from right table are displayed
-- matching rows from the left are displayed
-- if no match then null is populated.
select *
from students s
right join enrollments e
on s.s_id = e.s_id
right join courses c
on c.c_id = e.c_id;

```

```

-- 5) Right null join:-
select *
from students s
right join enrollments e
on s.s_id = e.s_id
right join courses c
on c.c_id = e.c_id
where s.s_id is null;

```

```

-- 6) Cross join:-
-- Every row of table 1 is combined with every row
-- from table 2
-- If table 1 has 3 rows and table 2 has 4 rows
-- total rows = 12 rows as output!
select s.s_name, c.c_name
from courses c
cross join students s;

```

```

-- 7) Self join (not a seprate keyword)
-- a table joined to itself.

```

```

create table employees(e_id int primary key auto_increment,
e_name varchar(225) not null,
m_id int,
constraint fk_employee_manager foreign key(m_id)
references employees(e_id)

```

```
on delete set null on update cascade);
```

```
INSERT INTO employees (e_name, m_id) VALUES
-- Top Level Managers
('Raj Malhotra', NULL),      -- 1
('Neha Sharma', NULL),      -- 2
('Amit Verma', NULL),       -- 3

-- Employees under Raj (1)
('Karan Mehta', 1),         -- 4
('Simran Kaur', 1),         -- 5
('Deepak Yadav', 1),        -- 6

-- Employees under Neha (2)
('Ritika Sen', 2),          -- 7
('Mohit Bansal', 2),        -- 8
('Anjali Rao', 2),          -- 9

-- Employees under Amit (3)
('Saurabh Jain', 3),        -- 10
('Pooja Nair', 3),          -- 11

-- Lower Level (Reporting to mid-level managers)
('Harsh Patel', 4),         -- 12
('Nidhi Kapoor', 5),        -- 13
('Aditya Singh', 7),        -- 14
('Megha Gupta', 10);       -- 15
```

```
select * from employees;
```

```
select e1.e_id as manager_id,
e1.e_name as manager_name,
e2.e_id as employee_id,
e2.e_name as employee_name
from employees e1
join employees e2
on e1.e_id = e2.m_id;
```

```
-- 8) Full outer join(set operators):-
-- union
-- full outer join is not supported by MySQL by default
```

```
-- Natural join(supported by not recommended)
-- automatically joins columns with the same name
-- very risky in production.
```

```
select *  
from students  
natural join enrollments;
```